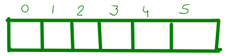
N Queen Type

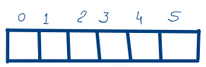


you have q=3 Queen where $q_0 > q_1 > q_2$ according to power

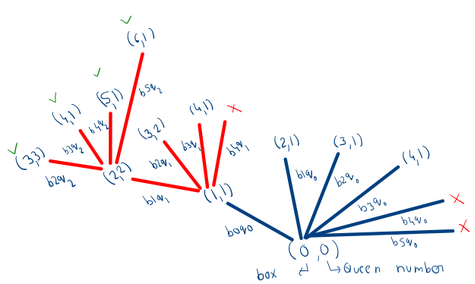
so place the queens in that boxes and give all combination

Ans→ as we have to find all the combination it is same as Combination type.

→ ∵ q_0 is more power so we have to place the q_0 first.



→ to place the all Queens we can place q_0 at index 0, 1, 2, 3 because at index 4 and 5 we can not place the rest Queens.




```
// tnq =Total number of Queen ,bno = box number,
public static int queencombination1D(boolean[] box, int tnq, int bno, int qpsf, String psf) {
    if (qpsf == tnq) {
        System.out.println(psf);
        return 1;
    }
    int cnt = 0;
    for (int bidx = bno; bidx < box.length; bidx++) {
        cnt += queencombination1D(box, tnq, bidx + 1, qpsf + 1, psf + "b" + bidx + "q" + qpsf + " ");
    }
    return cnt;

}
```

Queen Permutation 1D

→ same as permutation if a Queen used any box just marked it that it is used now and after call unmark this.

→ Code:

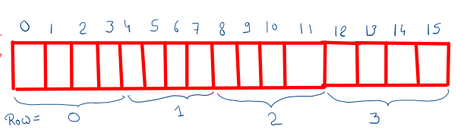
```
public static int queenpermutation1D(boolean[] box, int tnq, int qpsf, String psf) {
    if (qpsf == tnq) {
        System.out.println(psf);
        return 1;
    }
    int cnt = 0;
    for (int bidx = 0; bidx < box.length; bidx++) {
        if (!box[bidx]) {
            box[bidx] = true;
            cnt += queenpermutation1D(box, tnq, qpsf + 1, psf + "b" + bidx + "q" + qpsf + " ");
            box[bidx] = false;
        }
    }
    return cnt;

}
```

Queen Combination 2D

→ given an nxn matrix and total number of Queen find all the Combination of it

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

Convert it into 1D array



for row n/total Colum ex = 5/4 = 1

for Column n%.total Colum ex 5%.4 = 1

Code of 2D NQueen Combination

```
// nqueeen 2D
public static int queencombination2D(boolean[][] box, int tnq, int bno, int qpsf, String psf) {
    if (qpsf == tnq) {
        System.out.println(psf);
        return 1;
    }
    int cnt = 0;
    int n = box.length;
    for (int bidx = bno; bidx < n * n; bidx++) {
        int r = bidx / n;
        int c = bidx % n;
        cnt += queencombination2D(box, tnq, bidx + 1, qpsf + 1, psf + "(" + r + "," + c + ") ");
    }
    return cnt;

}
```

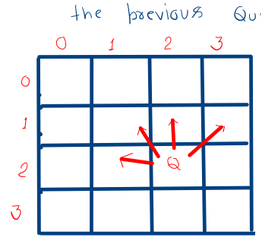
Nqueen permutation 2D

```
public static int queenpermutation2D(boolean[][] box, int tnq, int qpsf, String psf) {
    if (qpsf == tnq) {
        System.out.println(psf);
        return 1;
    }
    int cnt = 0;
    int n = box.length;
    for (int bidx = 0; bidx < n * n; bidx++) {
        int r = bidx / n;
        int c = bidx % n;
        if (!box[r][c]) {
            box[r][c] = true;

            cnt += queenpermutation2D(box, tnq, qpsf + 1, psf + "(" + r + "," + c + ") ");
            box[r][c] = false;
        }
    }
    return cnt;

}
```

Nqueen official

→ Same as Combination 2D just check where we are going to place the Queen is safe or not. If safe place the Queen if not check next place.

→ If the Queen is not able to place in complete row the the previous Queen is place at wrong place.



If we place a Queen we have to check in these 4 direction weather placing is safe or not

```
public static boolean isSafetoPlacequeen(boolean[][] box, int r, int c){
    int[][] dir = {{ 0, -1 }, { -1, -1 }, { -1, 0 }, { -1, 1 }, { 0, 1 }, { 1, 1 }, { 1, 0 }, { 1, -1 } };
    int n = box.length;
    for (int d = 0; d < dir.length; d++) {
        for (int rad = 1; rad <= n; rad++) {
            int x = r + rad * dir[d][0];
            int y = c + rad * dir[d][1];
            if (x >= 0 && y >= 0 && x < n && y < n) {
                // check weather the queen is already placed or not
                if (box[x][y]) {
                    return false;
                }
            } else {
                break;
            }
        }
    }
    return true;

}
```

```
public static int queen01permutation(boolean[][] box, int tnq, int qpsf, String psf) {
    if (qpsf == tnq) {
        System.out.println(psf);
        return 1;
    }
    int cnt = 0;
    int n = box.length;
    for (int bidx = 0; bidx < n * n; bidx++) {
        int r = bidx / n;
        int c = bidx % n;
        if (isSafetoPlacequeen(box, r, c)) {
            box[r][c] = true;

            cnt += queen01permutation(box, tnq, qpsf + 1, psf + "(" + r + "," + c + ") ");
            box[r][c] = false;
        }
    }
    return cnt;

}
```

NQueen01 permutation

→ We can rearrange the Queen placing so we need to check to the all direction weather it is safe to place or not so

```
public static boolean isSafetoPlacequeen_permutation(boolean[][] box, int r, int c) {
    int[][] dir = { { 0, -1 }, { -1, -1 }, { -1, 0 }, { -1, 1 }, { 0, 1 }, { 1, 1 }, { 1, 0 }, { 1, -1 } };
    int n = box.length;
    for (int d = 0; d < dir.length; d++) {
        for (int rad = 1; rad <= n; rad++) { // radious 1 se start isiliye he because 0 per wo
khud ko kill ka check
                                            // karega so
            int x = r + rad * dir[d][0];
            int y = c + rad * dir[d][1];
            if (x >= 0 && y >= 0 && x < n && y < n) {
                // check weather the queen is already placed or not
                if (box[x][y]) {
                    return false;
                }
            } else {
                break;
            }
        }
    }
    return true;

}

public static int queen01permutation(boolean[][] box, int tnq, int qpsf, String psf) {
    if (qpsf == tnq) {
        System.out.println(psf);
        return 1;
    }
    int cnt = 0;
    int n = box.length;
    for (int bidx = 0; bidx < n * n; bidx++) {
        int r = bidx / n;
        int c = bidx % n;
        if (!box[r][c] && isSafetoPlacequeen_permutation(box, r, c)) {
            box[r][c] = true;

            cnt += queen01permutation(box, tnq, qpsf + 1, psf + "(" + r + "," + c + ") ");
            box[r][c] = false;
        }
    }
    return cnt;

}
```